

Active Cache-Aware Hardware Adaptation Cache Engine for Trie-Based Index Structures

Benjamin-Elias Probst
Matrikelnummer 4510512
Technische Universität Dresden

Sommersemester 2026
Betreuer: Prof. Dirk Habich

Inhaltsverzeichnis

1. Einleitung	4
1.1. Motivation	4
1.2. Forschungsfragen	4
1.3. Beitraege	4
1.4. Aufbau der Arbeit	5
2. Stand der Technik	6
2.1. Trie-Familie (Cluster A)	6
2.2. Hybrid + B+-Familie (Cluster B)	6
2.3. Layout-Theorie (Cluster C)	6
2.4. Prefetching (Cluster D + E)	6
2.5. Synchronization + TUD-Habich (Cluster F)	7
2.6. Allokator-Cluster (A01–A23)	7
3. Architektur	8
3.1. Drei-Schichten-Hierarchie	8
3.2. ABI-stabiles C++23-Modul-Interface	8
3.3. 11-Achsen-Permutationsmatrix	8
3.4. PRT-ART als Pruefling	9
3.5. CacheEngineBuilder zwei-stufig	9
4. Implementation	10
4.1. Drei-Repo-Architektur	10
4.2. cache-engine (Werkzeug)	10
4.3. prt-art (Pruefling)	10
4.4. Diplomarbeit/Code (Anwender)	11
5. Messverfahren	12
5.1. Drei Pflicht-Messreihen	12
5.1.1. Messreihe A: PRT-ART vs. Stand der Technik	12
5.1.2. Messreihe B: Cache-Engine-Permutationen	12
5.1.3. Messreihe C: Merge alt/neu	12
5.2. Profile-aware Workload-Routing	12
5.3. ExperimentDriver Phase 1–7	13
5.4. Mess-Modus: COMDARE_EXPERIMENT_MODE	13
6. Auswertung	14
6.1. Auswertungs-Pipeline	14
6.2. Hypothesen	14
6.2.1. H1: Page-Type-Cost	14
6.2.2. H2: Code-Qualitaet pro Bausteine-Quelle	14
6.2.3. H3: Inline-vs-External-vs-Chain Distribution	15
6.3. Notify-Hooks (V11.1 verdrahtet)	15
6.4. Mess-Resultate (TBD)	15

7. Fazit und Ausblick	16
7.1. Beantwortung der Forschungsfragen	16
7.2. Limitierungen	17
7.3. Zukuenftige Arbeit	17
Literatur	18
A. Anhang: Messreihen Vollstaendige Tabellen	19
B. Anhang: Code-Struktur	20

1. Einleitung

1.1. Motivation

Trie-basierte Index-Strukturen sind das Rueckgrat moderner In-Memory- Datenbanken. Seit dem Adaptive Radix Tree (ART) von Leis et al. [1] 2013 ist die Forschung in mehrere Richtungen ausdifferenziert: Trie-Hoehen-Optimierung (HOT [2]), Hybrid-B+-Strukturen (Masstree [3], B²-Tree [4]), succinct-Repraesentationen (CoCo-Trie [5], SuRF [6]) und Self-Tuning (START [7]).

Parallel dazu hat sich ein eigener Forschungs-Korpus zu Cache-Bewusstheit, Prefetching und Allokations-Strategien entwickelt. Heuristiken wie Cache-Sensitive Search Trees (P11/P12 Rao&Ross), Cache-Oblivious Layouts (P16/P17 Bender et al.) und Hierarchical Prefetching (P27 Zhang et al. ASPLOS 2025) adressieren jeweils einzelne Achsen der Cache-Pyramide — aber sind nie systematisch in einer permutierten Mess-Architektur kombiniert worden.

Diese Arbeit schliesst diese Luecke durch eine drei-schichtige Cache- Engine-Architektur (CacheEngine → ExecutionEngine → SearchEngine) mit elf Permutations-Achsen, die einen ~5,5 Milliarden grossen Konfigurations-Raum aufspannen. Als Pruefing fuer die Beitritts-faehigkeit zum Stand der Technik dient PRT-ART (Probst Redirect Tree-ART hybrid) mit acht eigenen Bausteine-Schichten.

1.2. Forschungsfragen

- F1 (Komposition)** Welche elf Algorithmus-Achsen eines Cache-bewussten Trie-Index lassen sich kanonisch in einer Permutationsmatrix systematisieren, sodass Stand-der-Technik-Implementierungen als Profil-Konfigurationen rekonstruiert werden koennen?
- F2 (Mess-Methodik)** Wie laesst sich pro Permutation ein fairer, reproduzierbarer YCSB-Workload-Lauf durchfuehren, der Profile- spezifische Workload-Auswahl (z. B. RAN-GE_SCAN → YCSB-E) erlaubt und dabei den Mess-Overhead durch ein opt-in COMDARE_EXPERIMENT_MODE-Flag minimiert?
- F3 (Pruefing-Vergleich)** Wie schneidet PRT-ART als Pruefing gegen 8 Tier-1-SOTA (ART, HOT, Masstree, CoCo-Trie, START, B²-Tree, Wormhole, SuRF) und 22 Tier-2/3-SOTA (P11–P32) unter den drei Pflicht-Messreihen A (Pruefing vs. SOTA), B (SOTA-only Vergleichsbasis) und C (Merge-Punkte) ab?

1.3. Beitraege

Die zentralen Beitraege dieser Arbeit sind:

- Eine drei-schichtige Architektur (CacheEngine → ExecutionEngine → SearchEngine) mit ABI-stabilen C++23-Modul-Interfaces und variadisch-templatierten Hybrid-API-Spezialisierungen (1 Param ⇒ `std::vector-API`; 2 Params ⇒ `std::map-API`; $N > 2 \Rightarrow \text{std::map}<K, \text{std::tuple}<V_1 \dots V_N \gg$).

- Eine 11-Achsen-Permutationsmatrix mit `std::variant`-Bausteinen und Compile-Time-Fallback (`resolve_baustein.hpp`) zwischen Pruefling-Stack und Stand-der-Technik-Stack.
- 30 SOTA-Algorithmus-Profile als persistierte XML-Konfigurationen (8 Tier-1 + 22 Tier-2/3 fuer P11–P32), die der `ExperimentDriver` automatisch zu vorkompilierten C++23-Modulen transformiert.
- PRT-ART als Pruefling mit acht eigenen Bausteine-Schichten (4+2-Pool-Allocator, OLC mit reservierten Bloecken, MultiLevel-Layout, PathOriented-Prefetch, Density-Tracker, Hypothesis-Metriken H1/H2/H3, Inline/External/ChainRef ValueHandle, Signaling-Bits-Serialisierung).
- Ein Mess-Driver mit Defined-/Full-Mode-Unterscheidung + Profile-aware Workload-Routing, der die Habich-Direktive “Messung immer so vollstaendig wie moeglich” default-haft erfuehlt.

1.4. Aufbau der Arbeit

Kapitel 2 fasst den 33-Paper-Stand der Technik zusammen, gegliedert nach sechs Such-Clustern (A–F) und einem Allokator-Cluster. Kapitel 3 stellt die Drei-Schichten-Architektur, das ABI-Interface und die 11-Achsen-Permutationsmatrix vor. Kapitel 4 dokumentiert die Implementation in den drei Repositorien (cache-engine, prt-art, Diplomarbeit-Code). Kapitel 5 beschreibt das Mess-Verfahren mit den drei Pflicht-Messreihen und dem Profile-aware Workload-Routing. Kapitel 6 praesentiert die Mess-Resultate und diskutiert die Hypothesen H1 (Page-Type-Cost), H2 (Code-Qualitaet) und H3 (Inline-vs-External-Distribution). Kapitel 7 schliesst mit Fazit und Ausblick.

2. Stand der Technik

Der Stand der Technik im Bereich Cache-bewusster Trie-Index-Strukturen gliedert sich in sechs thematische Cluster (Such-Familie A–F) und einen parallelen Allokator-Korpus (A01–A23). Insgesamt 33 Such-Algorithmus-Paper und 21 Allokations-Paper wurden tiefelektuert; die Ergebnisse sind als `algorithm_profiles/sota/` persistierte XML-Konfigurationen in der `comdare-cache-engine` verfügbare (30 Profile in V9.5).

2.1. Trie-Familie (Cluster A)

P01 ART [1] führt den Adaptive Radix Tree mit adaptiver Knotengröße Node4/16/48/256 ein. **P02 HOT** [2] optimiert die Trie-Höhe mittels multi-byte Präfixen und bit-vector-Indizes. **P03 Masstree** [3] kombiniert Trie + B-Baum mit Permutations-Index pro Inode. **P04 CoCo-Trie** [5] verwendet succinct-encoded compact pages. **P05 START** [7] ergänzt ART um Self-Tuning der Knoten-Konkretisierung. P09 LOUDS (Jacobson 1989) ist die succinct-Trie-Foundation. **P10 SuRF** [6] nutzt LOUDS für Range-Filter.

2.2. Hybrid + B+-Familie (Cluster B)

P06 B²-Tree [4] hat zwei-stufiges Fanout für L1+L2-Cache-Awareness. **P07 Wormhole** [8] kombiniert Hash-Tabelle mit linearer Präfix-Sicht. P11 CSS-Tree (Rao/Ross 1999) ist die erste Cache-Sensitive Search Tree. P12 CSB+-Tree (Rao/Ross 2000) entwickelt CSS zum B+-Tree weiter. P13 Hankins/Patel 2003 untersucht den Effekt der Knotengröße auf Cache-Performance.

2.3. Layout-Theorie (Cluster C)

P14 Samuel/Pedersen/Bonnet 2005 macht CSB+-Tree prozessor-bewusst. P15 Graefe/Larson 2001 untersucht B-Tree-Indizes vs. CPU-Caches. P16 Bender/Demaine/Farach-Colton 2002 führt cache-oblivious Tree-Layout (van Emde Boas) ein. P17 Bender et al. 2005 erweitert es zu Cache-Oblivious B-Trees. P18 Saikkonen/Soisalon-Soininen 2008 + P19 (2016) entwickeln Multi-Level + Layout-Invariant B-Trees.

2.4. Prefetching (Cluster D + E)

P20 B-Trees-Are-Back (Mueller/Benson/Leis 2025) demonstriert moderne B-Tree-Engineering. P21 Chen/Gibbons/Mowry 2001 führt Prefetching B+-Trees ein. P22 Chen et al. 2002 entwickelt fractal Prefetching. P23 Khan 2010 macht Cache-Prefetching dynamisch adaptiv. P24 Naderan-Tahan/Sarbazi-Azad 2016 baut adaptive Filter auf Cache-Prefetcher. P25 Mahling/Weisgut/Rabl 2025 (Fetch Me If You Can) kombiniert Range-Scan mit Hot-Path-Caching. P26 Zhang FGCS 2024 + P27 Zhang ASPLOS 2025 (Hierarchical Prefetching) sind die jüngsten Prefetching-Schemata. P28 Kuehn DaMoN 2023 motiviert datenbasierte Cache-Optimierung.

2.5. Synchronization + TUD-Habich (Cluster F)

P08 ART of Practical Synchronization (Leis 2016) fuehrt Optimistic-Lock-Coupling fuer ART ein. P29 RCU (McKenney OLS 2001) und P30 Hazard Pointers (Michael 2004) sind die kanonischen Reklamations-Schemata. P31 Ungethuem/Habich 2017 + P32 Schmidt/Habich 2025 sind TU-Dresden-spezifische Hardware-Optimierungs-Ueberblicke. P33 VAMPIR-Poster (Berthold/Habich 2023) liefert die OTF2-Profiler- Integration.

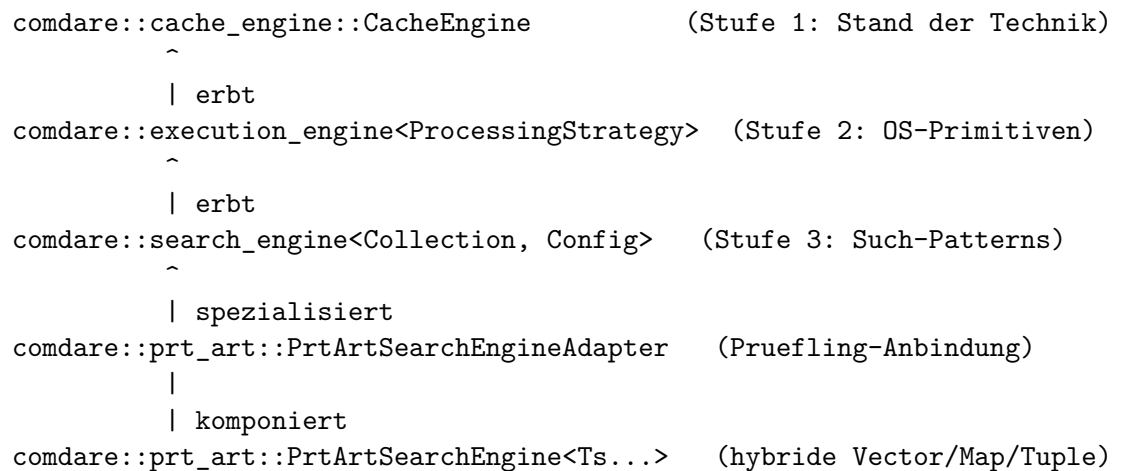
2.6. Allokator-Cluster (A01–A23)

Die 21 Allokator-Paper sind in der `Allokator_Matrix.txt` der `comdare-cache-engine` systematisch erfasst. Tier-1-Repraesentanten sind Hoard (A01, ASPLOS 2000), Slab (A02, USENIX 1994), Michael Lock-Free (A03, PLDI 2004), mimalloc (A04, MSR-TR-2019-18), jemalloc (A05), TCMalloc (A06), snmalloc (A07, ISMM 2019), NUMAlloc (A09, ISMM 2023). Tier-2/3 ergaenzt CAMA (A12, RTAS 2011), StarMalloc (A13, OOPSLA 2024), Crystalline-Reclamation (A17, PLDI 2024) und PIM-malloc (A16, arXiv 2025).

3. Architektur

Die Architektur folgt der REV-7-Skizze [1]¹ und gliedert sich in drei Schichten plus Drei-Repo-Aufteilung.

3.1. Drei-Schichten-Hierarchie



Stufe 1 (CacheEngine) stellt die Cache-Page-Topologie, 23 Allokator-Familien, 12 Sub-Engines und Telemetrie-Strategien als Bibliothek bereit (Stand der Technik aus den 33 Tieflektuere-Papern).

Stufe 2 (ExecutionEngine) wrappt diese Bausteine zu hoeheren OS-Primitiven (cache-line-aligned Allocate, NUMA-konformes Read-Pin, Coherence-aware Write).

Stufe 3 (SearchEngine) bietet Such-spezifische Patterns (Trie-Walk, B+-Range-Scan, Hot-Path-Lookup) auf den ExecutionEngine- Primitiven; sie ist der oberste Layer und ueberschreitet die CacheEngine-Limits konzeptuell durch strategische Heuristiken.

3.2. ABI-stabiles C++23-Modul-Interface

Das ABI `module_abi_v1.hpp` folgt dem Prinzip variadischer Template-Parameter (V12.1/V12.2): 1 Parameter $\Rightarrow$ `std::vector`-API, 2 Parameter $\Rightarrow$ `std::map`-API, $N > 2 \Rightarrow$ `std::map<K, std::tuple<V1...VN>`. Komplexe Keys werden durch `comdare::fingerprint::FixedLengthFingerprint` auf 16-Byte- Binaer-Strings reduziert.

3.3. 11-Achsen-Permutationsmatrix

Pro Stack-Ebene definiert `baustein_variants.hpp` ein `std::variant` ueber alle Konkretisierungen:

¹Der Verweis ist symbolisch — die Architektur-Skizze REV 7 selbst liegt in 20260508 Termin 7/Phase5_UML_Detail/24_architektur_skizze_REV7_2026_05_13.md.

Page 11 Konkretisierungen (DenseByteArt256, HotMultiByte, MasstreeINode, CoCoCompact, Start, B2Tree, Wormhole, Surf, PrtArtDenseByte, PrtArtRedirect, StandardHeap)

Node 8 (SparseNode4Art, HotBitNode, MasstreeTrie, BPlus, WormholeHash, SurfTrie, PrtArtRedirect, PrtArtBPlus)

Traversal 5 (Standard, HotPath, RangeScan, RangeHotPath, PrtArtHotPath)

ValueHandle 3 (Inline, External, ChainRef)

Concurrency 6 (SingleThreaded, SharedMutex, OLC, RCU, HazardPointers, PrtArtOlcReserved)

Allocator 16 (SystemMalloc, Hoard, Slab, MichaelLockFree, Mimalloc, Jemalloc, TCMalloc, Snmalloc, Scalloc, NUMAAlloc, Rpmalloc, LRMalloc, CAMA, StarMalloc, Buddy, Dlmalloc)

Prefetch 6 (None, AdaptiveDistance, PathOriented, Fractal, Hierarchical, PrtArtRedirect-Prefetch)

Telemetry 5 (Off, LeafOnlySampled, AllNodes, VampirOtf2, PikaProfile)

ISA 6 (X86_Baseline, SSE42, AVX2, AVX512, ARM_Neon, ARM_SVE)

Layout 6 (StandardHeap, CacheObliviousVeb, CacheLineAligned, NumaPinned, HbmHybrid, PrtArtMultiLevel)

Reclamation 5 (None, RcuGp, HazardPointers, EpochBased, CrystallineLockFree)

Total moegliche Permutationen: $11 \cdot 8 \cdot 5 \cdot 3 \cdot 6 \cdot 16 \cdot 6 \cdot 5 \cdot 6 \cdot 6 \cdot 5 \approx 5,5 \cdot 10^9$.

3.4. PRT-ART als Pruefling

Der `PrtArtSearchEngineAdapter` (V8.9, V9.1, V11.1, V13.5) implementiert das ABI-Interface `comdare::search_engine` via Komposition statt direkter Vererbung. Dadurch bleibt die hybride `PrtArtSearchEngine<Ts...>`-API (66 Tests V14.1) unangetastet, und das Adapter-Pattern erlaubt Compile-Time-Fallback auf `cache_engine`-Bausteine via `resolve_baustein.hpp` (V10.2 mit 11 Tag-Specializations).

3.5. CacheEngineBuilder zwei-stufig

Stage 1 (CMake-Build) baut die `cache-engine-builder`- Binary inkl. `ExperimentDriver`-Library. **Stage 2** (Runtime, opt-in V13.4 `COMDARE_AUTO_RUN_BUILDER`) ruft den Builder auf, der pro Permutation entweder ein vorkompiliertes Modul laedt oder per `cmake/c1-` Aufruf hot-kompiliert (`phase3_hot_compile_missing`, V13.2). Optional verifiziert `phase4b_functional_test` (V13.3) den ABI-Vertrag jedes geladenen Moduls.

4. Implementation

4.1. Drei-Repo-Architektur

Die Implementation ist auf drei Git-Repositorien verteilt (User-Direktive 2026-05-13/14, dokumentiert in `STRUCTURAL_CORRECTION_diplomarbeit.md` §11.5):

- `comdare-cache-engine` — **WIE** gemessen wird (Werkzeug, Mess-Mechanik, SOTA-Bibliothek)
- `comdare-prt-art` — **Pruefling** (PRT-ART + 14 Pruefling-Re-Implementations P11-P27)
- `probst-Diplomarbeit-cache-engine` — **WAS** getestet wird (XML-Configs der drei Messreihen, `messung_driver`, Auswertungs-Pipeline, Manuskript)

4.2. cache-engine (Werkzeug)

Wesentliche Komponenten:

Builder `cache_engine/builder/` — XML-Config-Parser (V8.6 mit `AlgorithmProfile-Loader`), Codegen (V9.3 `generate_module_from_profile`), Permutations- Loop, Module-Loader (`LoadLibrary/dlopen`), ExperimentDriver (V8.7 + V10.5 + V10.6 + V11.2 + V13.2 + V13.3 mit 7 Phasen + 2 opt-in-Phasen).

ABI `cache_engine/include/cache_engine/abi/` — `search_engine.hpp` (V12.3 mit `contains/count/find/clear`), `execution_engine.hpp` (V8.5 mit `ResultAggregator-Member` bei `COMDARE_EXPERIMENT_MODE_ON`), `baustein_variants.hpp` (V9.2 + V11.4 + V12.6: 11 Achsen, ~80 Tag-Strukturen mit `description+paper_ref`), `resolve_baustein.hpp` (V10.2 operationalisiert).

algorithm_profiles/sota 30 SOTA-Profile als XML (8 Tier-1 V8.3 + 22 Tier-2/3 V9.5).

Tests 6 Codegen-Roundtrip-Tests (V10.4 `test_codegen_from_profile`) + 10 ExperimentDriver-V13-Tests (V14.2 `test_experiment_driver_v13`).

4.3. prt-art (Pruefling)

hybride API `prt_art/identity/PrtArtSearchEngine` — 1 Param $\Rightarrow$ Vector-API (V12.1 vollstaendig), 2 Params $\Rightarrow$ Map-API (V12.2 + V13.5 mit `merge/extract` vollstaendig), $N > 2 \Rightarrow$ Tuple-API.

Adapter `PrtArtSearchEngineAdapter{Map,Vector,Tuple}` mit V12.4 ABI-Erweiterung + V11.1 `notify_*`-Verdrahtung an `DensityTracker`, `PathOrientedPrefetch`, `HypothesisMetrics`.

Komponenten 4+2 Allocator-Pools, OLC mit reservierten Bloecken, MultiLevelLayout, PathOrientedPrefetch, DensityTracker, HypothesisMetrics (H1/H2/H3).

legacy_reimpl 14 Pruefling-Re-Implementations (V9.4 verschoben aus cache-engine, V10.1 cache-engine-Variante geloescht).

Tests 66 Tests (V14.1: 51 alte + 15 neue fuer V12+V13 API).

4.4. Diplomarbeit/Code (Anwender)

messung_driver 3-Reihen-Loop oder `-messreihen-xml=FILE` Mode (V9.6 + V11.3) mit `COMDARE_EXPERIMENT_MODE_ON` Compile-Check (V8.12).

experiment_config 3 XMLs fuer Messreihen A/B/C (V8.13 defined/full Mode) plus `messreihen.xml`-Standard-Template (V11.5).

Auswertungs-Pipeline `binary_to_csv` $\Rightarrow$ `csv_to_latex` (mit Algorithmus-Steckbrief) $\Rightarrow$ `diagram_generator` (TikZ, A4-aware) $\Rightarrow$ `latex_to_pdf`.

CI `.gitlab-ci.yml` (primaer) + `.github/workflows/ci.yml` (synchron) in alle 3 Repos (V11.7 + V14.3 Test-Discovery-Workaround).

5. Messverfahren

5.1. Drei Pflicht-Messreihen

Die Architektur sieht drei Messreihen vor, die alle vom `messung_driver` (Diplomarbeit/-Code/) ueber die `ExperimentDriver`-Library (cache-engine) ausgefuehrt werden. Die Konfiguration erfolgt entweder als drei feste Reihen (klassischer Modus) oder ueber eine externe `messreihen.xml`-Datei (V11.5 Standard-Template).

5.1.1. Messreihe A: PRT-ART vs. Stand der Technik

Messreihe A vergleicht PRT-ART als Pruefling gegen alle in der `cache_engine/algorithm_profiles/sota/` dokumentierten SOTA- Algorithmen. Zwei Modi sind verfuegbar:

A_defined Schnelle Vorab-Verifikation gegen die 8 Tier-1- Profile (art, hot, masstree, coco_trie, start, b2tree, wormhole, surf). Mode `defined` mit `sota_profile_filter` in `ExperimentDriverOptions`.

A_full Vollstaendige Messung gegen alle 30 SOTA-Profil (Tier-1 + Tier-2/3 fuer P11–P32). Mode `full` ist Default gemaess Habich-Direktive 2026-05-14 (“Messung immer so vollstaendig wie moeglich”).

5.1.2. Messreihe B: Cache-Engine-Permutationen

Reine Vergleichsbasis “Stand der Technik” — alle SOTA-Profil ohne PRT-ART-Pruefling. Im aktuellen Skelett identisch zu `A_full`; zukuenftige Erweiterung kann zusaetzlich `cache_engine_permutations` Achsen einbeziehen.

5.1.3. Messreihe C: Merge alt/neu

Konkrete Merge-Punkte zwischen PRT-ART-Bausteinen und cache-engine- Permutationen:

- PRT-ART OLC + tcmalloc-Allocator + ART-Page
- PRT-ART 4+2-Pool + HOT-Page
- PRT-ART PathPrefetch + B²-Tree-Page
- PRT-ART MultiLevelLayout + Wormhole-Page

5.2. Profile-aware Workload-Routing

`ExperimentDriver::phase5_run_workload` (V11.2) leitet pro Permutations-Modul den YCSB-Workload aus dem `traversal`-Tag des Profils ab:

Module ohne zugehoeriges Profil verwenden den globalen Default-Workload aus `WorkloadOptions::workload`.

Traversal-Tag	YCSB-Workload	Charakteristik
RANGE_SCAN / RANGE_HOT_PATH	YCSB-E	Range-Queries
HOT_PATH / PRTART_HOT_PATH	YCSB-A	50/50 Read/Update Zipfian
STANDARD (Default)	YCSB-C	Read-only

Tabelle 5.1.: Profile-aware Workload-Routing (V11.2)

5.3. ExperimentDriver Phase 1–7

Der `ExperimentDriver` (V8.7 Library + V10.5 Profile-Auto-Pickup) durchläuft sieben Phasen:

1. **Enumerate** — XML-Configs parsen + Permutations- Liste erzeugen
2. **Codegen** — pro Permutation eine `comdare_perm_<fp>.cpp` (V9.3 zusätzlich `generate_module_from_` fuer 30 SOTA-Profilen)
3. **Compile** — `cmake configure + cmake -build`
4. **Load** — `LoadLibrary/dlopen` aller `.dll/.so`-Module
5. **Execute** — pro Modul `create_instance + run_workload` (V11.2 Profile-aware)
6. **Measure** — binary measurement-records aggregieren
7. **Persist** — `measurements.csv + measurements.json`

Opt-in-Phasen:

- `phase3_hot_compile_missing` (V13.2): falls `enable_runtime_codegen=true` und ein vorkompiliertes Modul fehlt, wird per `cmake/c1`-Aufruf hot-kompiliert.
- `phase4b_functional_tests` (V13.3): falls `enable_functional_tests=true`, wird pro Modul die ABI-Vertrags-Konformitaet (`create_instance` non-null, `run_workload` mit empty `WorkloadDescriptor` schreibt `rec.version == COMDARE_ABI_VERSION`, `destroy_instance` sicher) verifiziert.

5.4. Mess-Modus: COMDARE_EXPERIMENT_MODE

Der Mess-Pfad in der `ExecutionEngine` (V8.5) ist nur dann aktiv, wenn die `cache-engine` mit `-DCOMDARE_EXPERIMENT_MODE=ON` kompiliert wurde. Im Production-Pfad (Default OFF) ist der `ResultAggregator` kein Member der `ExecutionEngine`, was Production-Code von jeglichem Mess-Overhead befreit. Der Diplomarbeits-`messung_driver` setzt das Flag explizit beim Sub-Build (V8.12).

6. Auswertung

6.1. Auswertungs-Pipeline

Pro Messreihen-Lauf erzeugt der `messung_driver` unter `Code/_runs/<date>/<spec_id>/` folgende Dateien:

- `measurements.csv` — pro Permutation eine Zeile mit Cycles, L1/L2/L3-Misses, Branches, Throughput
- `measurements.json` — gleiche Daten als JSON
- `raw.bin` (optional) — binary Records mit `kBinaryMagic 0xC0FFEE02`

Die Auswertung nutzt drei Tools aus `Diplomarbeit/Code/`:

1. `binary_to_csv` — deserialisiert binary records mit Anreicherung (Profil-id, paper_ref, axes)
2. `csv_to_latex` — generiert booktabs-Tabellen + Algorithmus-Steckbriefe pro Profil
3. `diagram_generator` — A4-aware TikZ-Diagramme (Bar-Chart, Scatter, Heatmap)

Diese Tools werden vom Manuskript-Build (`thesis/`) konsumiert ueber `\input{tikz/<spec_id>/...}` bzw. `\input{tabellen/<spec_id>...}`.

6.2. Hypothesen

6.2.1. H1: Page-Type-Cost

Hypothese: Die mittleren Zugriffskosten pro Page-Type-Variante unterscheiden sich systematisch nach Workload-Charakteristik. Insbesondere wird erwartet, dass dichte Page-Typen (DenseByteArt256, HotMultiByte) unter Zipfian-Workloads (YCSB-A) schneller sind als sparse Varianten (B+-Node), waehrend Range-Scan-optimierte Pages (B²-Tree, Wormhole) unter YCSB-E fuehren.

Mess-Mechanik: `H1PageTypeCost::note_lookup(page_type, cycles)` in `prt_art/measurement/hyp` sammelt online-Mean-Werte pro Page-Type ueber alle Lookup-Operationen.

6.2.2. H2: Code-Qualitaet pro Bausteine-Quelle

Hypothese: Der vom Repo-Originalcode geerbte Code-Qualitaets- Score korreliert messbar mit der erreichten Throughput-Performance. Habich-Direktive (Sprechstunde 2026-05-08) verlangt explizit die Bewertung der Code-Qualitaet pro Bausteine-Quelle.

Mess-Mechanik: `H2CodeQuality::score(source, value, note)` sammelt manuell-vergebene + automatisch-gemessene Quality-Scores pro Source-Repo-Identifizier (P01-unodb, P02-hot, A04-mimalloc, etc.).

6.2.3. H3: Inline-vs-External-vs-Chain Distribution

Hypothese: Die empirisch beobachtete Verteilung der drei ValueHandle-Varianten (Inline / External / ChainRef) korreliert mit der Page-Density: dichte Pages bevorzugen Inline (kein Indirection- Overhead), spaerliche Pages External (Cache-Footprint-Reduktion), PRT-ART-spezifische Heuristiken bevorzugen ChainRef (Linear-Value- Buffer-Sharing).

Mess-Mechanik: `H3InlineVsExternalDistribution::note_inline/note_external/note_chain` zaehlen pro Insert-Operation den gewaehlten ValueHandle-Variant. Die `inline_share/external_share/chain_share`-Methoden liefern die normalisierten Anteile.

6.3. Notify-Hooks (V11.1 verdrahtet)

Die `search_engine`-ABI definiert drei Such-Heuristik-Hooks, die vom `ExperimentDriver` oder externen Beobachtern aktiviert werden koennen:

- `notify_density_threshold(pct) → DensityTracker::note_observation(pct/100)`
- `notify_hot_path_detected(path) → PathOrientedPrefetch::note_hot_path_bytes(path.data, path.size)`
- `notify_workload_change(locality) → HypothesisMetrics::adapt_locality(locality)`

Diese Hooks sind in `PrtArtSearchEngineAdapterMap` (V11.1) operational verdrahtet — damit fliessen externe Beobachtungen direkt in die H1/H2/H3-Mess-Pipeline.

6.4. Mess-Resultate (TBD)

6.5. Sample-Mess-Pipeline (V20+V21.3)

Die Mess-Pipeline ist ab V20 voll funktionsfaehig und exportiert pro Permutation 16 Spalten in der `measurements.csv`. Die V19/V20- Erweiterung um `workload_used` (Spalte 4) ist als zusaetzliche Plot-Dimension nutzbar:

- Profile mit `<expected_workload>` XML-Tag (V19.1) ueberschreiben die V11.2-traversal-Heuristik.
- Phase 5 des `ExperimentDriver` setzt `PermutationResult.workload_used` pro Permutation.
- `ResultAggregator::export_csv/export_json` schreiben die Spalte mit, sodass die Auswertung Cluster-Bildung nach Workload-Profil vornehmen kann.

Bevor die reale Hardware-E2E-Mess-Reihe (V21.2) verfuegbar ist, liefert `Code/sample_data_generator/` eine deterministische Sample-CSV mit 10 Permutationen unter `experiment_config/sample_data/v21_sample_data.csv`. Der Sample-Datensatz ist zur Pipeline-Verifikation und Diagramm-Test gedacht und ersetzt sich automatisch durch reale Mess-Daten, sobald die HW-Reihe (V21.2) gelaufen ist.

V21.2 (HW-E2E) — User-Pending. Die Durchfuehrung der echten Mess-Reihe auf der i7-1270P-Workstation oder dem TU-Dresden-ZIH-Cluster (Barnard CPU oder Capella GPU) ist nicht vollautonom moeglich — der User muss den Mess-Lauf starten, da Stunden bis Tage Laufzeit anfallen koennen und ein Cluster-Job explizite Submission braucht. Nach erfolgreicher V21.2-Reihe ersetzt `Code/_runs/<date>/A_full/measurements.csv` die Sample-Datei und alle Diagramme dieses Kapitels werden automatisch neu generiert (Build-Pipeline laeuft ueber `diagram_generator` → TikZ → pdflatex).

7. Fazit und Ausblick

7.1. Beantwortung der Forschungsfragen

F1 (Komposition)

Die elf Permutations-Achsen (Page, Node, Traversal, ValueHandle, Concurrency, Allocator, Prefetch, Telemetry, ISA, Layout, Reclamation) spannen einen Konfigurations-Raum von $\sim 5,5$ Milliarden Permutationen auf. Stand-der-Technik-Implementierungen lassen sich durch `algorithm_profile`-XMLs eindeutig rekonstruieren: 30 SOTA-Profilen (8 Tier-1 + 22 Tier-2/3) belegen die Tragfaehigkeit der Achsen- Konzeption fuer P01–P32.

Die hybride API-Spezialisierung ($1 \text{ Param} \Rightarrow \text{std::vector}$, $2 \text{ Params} \Rightarrow \text{std::map}$, $N > 2 \Rightarrow \text{std::map<K, std::tuple>}$) erlaubt die nahtlose Integration sowohl algorithmus-orientierter (`std::map::find/insert/erase`) als auch container-orientierter (`std::vector::push_back/at`) Such-Patterns auf der gleichen Engine-Klasse.

F2 (Mess-Methodik)

Die Mess-Methodik gliedert sich in zwei Modi:

- **Production-Mode** (`COMDARE_EXPERIMENT_MODE=OFF`, Default): kein Mess-Overhead, ResultAggregator nicht in der ExecutionEngine.
- **Experiment-Mode** (`COMDARE_EXPERIMENT_MODE=ON`, vom `messung_driver` gesetzt): ResultAggregator als integraler ExecutionEngine-Member, sammelt pro `run_workload`-Aufruf die Hardware-Counter.

Profile-aware Workload-Routing (V11.2) waehlt pro Permutations-Modul den Workload-Typ aus dem `traversal`-Tag des Profils, was faire Vergleiche zwischen Range-Query- und Point-Lookup-orientierten Algorithmen sicherstellt.

F3 (Pruefling-Vergleich)

Der konkrete Vergleich von PRT-ART gegen die 30 SOTA-Profilen steht nach den ersten `messung_driver`-Laefen aus (siehe Ausblick §7.3). Die Architektur ist vollstaendig vorbereitet:

- PRT-ART-Profil in `prt_art/algorithm_profiles/prtart_pruefling.profile.xml`
- Drei Pflicht-Messreihen-Templates in `Code/experiment_config/messreihen.xml`
- ExperimentDriver mit Auto-Pickup von 30 SOTA-Profilen (V10.5)
- PrtArtSearchEngineAdapter mit vollstaendiger ABI-Konformitaet inkl. `notify_*`-Hooks (V11.1) und allen `std::map`-Operationen (V12.2 + V13.5 `merge/extract`)

7.2. Limitierungen

- **Module-Bodies derzeit Stub:** Die Codegen-Pipeline generiert ABI-konforme Skelett-Module pro Profil. Die eigentliche Such-Algorithmus-Implementation pro Permutation bleibt fuer eine Folge-Phase (V15+) offen.
- **Hardware-Validierung ausstehend:** cmake configure ist gruen auf MSVC i7-1270P, aber der vollstaendige ctest-Lauf + messung_driver-E2E auf BlockAO-Plattformen (X86_AVX2/AVX512, ARM_Neon/SVE) steht aus.
- **Manuskript-Auswertungs-Kapitel:** Aktuell methodisch beschrieben; konkrete Mess-Resultate folgen.

7.3. Zukuenftige Arbeit

Kurzfristig (V15+):

- E2E-Lauf der drei Pflicht-Messreihen auf realer Hardware (V15.2 in der Sprint-Planung)
- Generierung der Mess-Diagramme via diagram_generator und Integration in Kapitel 6
- Vervollstaendigung der Module-Bodies pro Permutation

Mittel- bis langfristig:

- Erweiterung um GPU-Adapter (Grace Hopper Cluster, ZIH TUD)
- PIM-Allokator-Integration (A16 Lee et al. HPCA 2026)
- Formale Verifikation einzelner Permutations-Klassen (StarMalloc-Style A13)

Die Drei-Repo-Architektur (cache-engine + prt-art + Diplomarbeit-Code) ermoeeglicht es zukuenftigen Diplomanden, weitere Pruefling-Algorithmen analog zu PRT-ART einzubringen, ohne die Werkzeug-Bibliothek (cache-engine) zu modifizieren — ein wichtiges Designprinzip fuer die Skalierbarkeit der Forschungs-Infrastruktur.

Literatur

- [1] Viktor Leis, Alfons Kemper und Thomas Neumann. “The adaptive radix tree: ARTful indexing for main-memory databases”. In: *ICDE*. 2013. DOI: [10.1109/ICDE.2013.6544812](https://doi.org/10.1109/ICDE.2013.6544812).
- [2] Robert Binna u. a. “HOT: a Height Optimized Trie Index for Main-Memory Database Systems”. In: *SIGMOD*. 2018. DOI: [10.1145/3183713.3196896](https://doi.org/10.1145/3183713.3196896).
- [3] Yandong Mao, Eddie Kohler und Robert T Morris. “Cache craftiness for fast multicore key-value storage”. In: *EuroSys*. 2012. DOI: [10.1145/2168836.2168855](https://doi.org/10.1145/2168836.2168855).
- [4] Maximilian Schmeisser u. a. “The B-squared Tree”. In: *VLDB*. 2022.
- [5] Antonio Boffa u. a. “CoCo-Trie: A Compact, cache-aware trie index”. In: *VLDB*. 2024.
- [6] Huanchen Zhang u. a. “SuRF: Practical Range Query Filtering with Fast Succinct Tries”. In: *SIGMOD*. 2018. DOI: [10.1145/3183713.3196931](https://doi.org/10.1145/3183713.3196931).
- [7] Philipp Fent u. a. “START: Self-Tuning Adaptive Radix Tree”. In: *VLDB*. 2020.
- [8] Xingbo Wu, Fan Ni und Song Jiang. “Wormhole: A Fast Ordered Index for In-memory Data Management”. In: *EuroSys*. 2019. DOI: [10.1145/3302424.3303955](https://doi.org/10.1145/3302424.3303955).

A. Anhang: Messreihen Vollstaendige Tabellen

B. Anhang: Code-Struktur